

最適化入門 — 定数畳み込みとピープホール

今日のゴール

自分のコンパイラに 2 つの最適化パスを追加して、生成される命令数を減らす。そして `fixed15` が全通し続けることで、意味を変えずに速くできたことを確認する。

1 この回の位置づけ

- バックエンド発展シリーズの第 1 回（選択制）。前提はコマ 8（関数呼び出し）まで。コマ 16 まで終えていれば `fixed15` で効果を測れる
- `mycc.py` は 1 行も書き換ええない。最適化は独立した 2 つのパスとして書き、ラッパー `optcc.py`（配布済み・完成品）が差し込む
- 編集するのは `fold.py` と `peephole.py` の 2 ファイル

2 最適化とは何か — 「速くする」の前に「壊さない」

いま自分のコンパイラは、`return 3 + 4;` に対して次のような命令を出している。

```
li a0, 3
addi sp, sp, -8
sd a0, 0(sp)
li a0, 4
ld a1, 0(sp)
addi sp, sp, 8
add a0, a1, a0
```

7 命令かけて、実行時に必ず 7 になる計算をしている。`li a0, 7` の 1 命令で済むはずである。

最適化とは、プログラムの観測できる振る舞いを変えずに、生成コードを速く（または小さく）することである。ここで「観測できる振る舞い」とは、この講義では次の 2 つを指す。

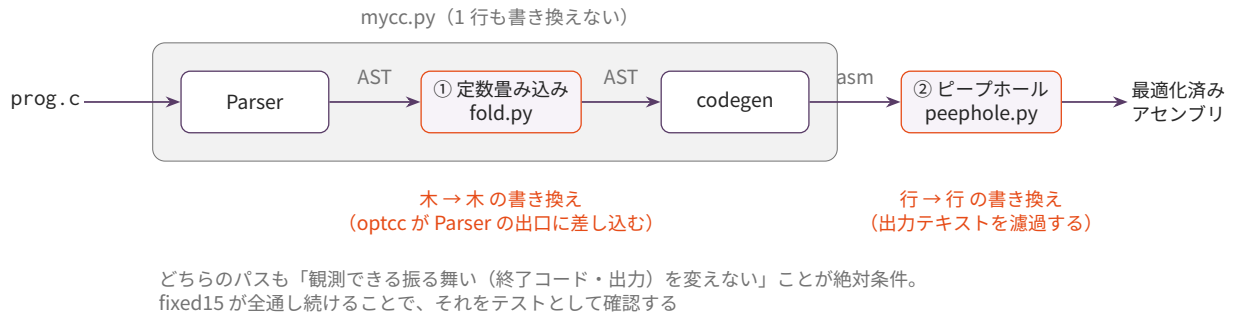


図 1 2つの最適化パスの差し込み位置

観測できるもの 確認方法

終了コード .ans ファイルとの比較

標準出力 .stdout ファイルとの比較

つまり既存のテストが全通し続けることが、最適化が正しいことの証明になる。fixed15 が、そのままこの回の安全網になる。

重要

最適化で最も多いバグは「速くならない」ではなく「壊れる」である。命令を 1 つ消すたびに fixed15 を回す習慣をつけること。テストが落ちたら、その置換は意味を変えている。

3 2つのパスをどこに差し込むか

最適化は、コンパイラのどの段階でも書ける。この回では対照的な 2 つを作る。

パス	入力 → 出力	見えるもの	見えないもの
① 定数畳み込み	AST → AST	式の構造・型	レジスタ・命令
② ピープホール	アセンブリ → アセンブリ	命令の並び	式の構造

同じ「無駄を消す」でも、見えている情報が違うので、消せる無駄が違う。上流ほど意味が分かるので大胆な変換ができ、下流ほど実際の命令に近いので細かい無駄が見える。実際のコンパ

イラも、この両方を持っている。

4 ① 定数畳み込み 一木を書き換える

コンパイル時に値が決まる部分式を、計算済みの `ND_NUM` に置き換える。

```
(add (num 1) (mul (num 2) (num 3))) → (num 7)
```

木を後行順（子が先）に処理するのがコツである。子を先に畳み込んでおけば、親では「両方の子が `ND_NUM` か」を見るだけで済む。

```
(1 + 2) * 3 → 子を畳む → (num 3) * 3 → 親を畳む → (num 9)
```

走査 (`fold_node`) と置き換えの判定 (`fold_expr`) はスケルトンに完成済みで、実装するのは演算の意味、つまり「この演算子を定数 2 つに適用したらいくつか」だけである。

4.1 落とし穴 1: C の除算は Python と違う

```
-7 // 2    # Python: -4 (床関数: 小さい方へ)
-7 / 2     # C:      -3 (0 方向へ切り捨て)
```

コマ 2・コマ 3 の資料で「負数を含む割り算・剰余は扱わない」と注意していたのは、この差があるからだ。畳み込みでは実行時と同じ答えを出さなければならないので、C の規則（0 方向へ切り捨て、剰余の符号は左辺と同じ）に合わせる必要がある。

`abs()` で計算してから符号を付け直すのが素直である。

4.2 落とし穴 2: 畳み込みではいけない場合

式	扱い	理由
<code>5 / 0</code>	畳み込まない (<code>None</code> を返す)	実行時エラーを消してはいけない
<code>1 << 64</code>	畳み込まない	シフト量が範囲外（結果は未定義）

式	扱い	理由
<code>a + 2 * 3</code>	<code>2 * 3</code> だけ畳む	変数の値はコンパイル時に分からない

「分からないものは触らない」 — 最適化の基本姿勢である。

4.3 落とし穴 3: 64bit で丸める

実行時のレジスタは 64bit なので、桁あふれもそのとおりに再現する必要がある。Python の整数は無限精度なので、`to_i64()`（配布済み）で毎回丸める。

5 ② ピープホール最適化 — 窓から覗いて置き換える

生成されたアセンブリを小さな窓（**peephole**）で覗き、既知のパターンを短い命令列に置き換える。文脈を見ないので実装が単純で、それでいてよく効く。

5.1 規則 A: `push / li / pop` の圧縮

コマ 3 以上の二項演算パターンは、右辺が**定数**のときに無駄が大きい。

```

addi sp, sp, -8      ┌
sd a0, 0(sp)         │ →  mv a1, a0
li a0, 1              │    li a0, 1
ld a1, 0(sp)         │
addi sp, sp, 8        └

```

右辺の計算が `li 1` だけなら、左辺を退避する必要がない。`a1` に移してから定数を読めば、同じ状態（`a1=左辺`、`a0=右辺`）が作れる。**5 命令が 2 命令になる**。`i = i + 1` のようなループ内の式に何度も現れるので、効果が大きい。

5.2 規則 B・C: 直後のラベルへ飛ぶジャンプの削除

```
j .L3      ← 消してよい
.L3:
```

飛ばなくても次の命令は同じなので、この `j` は不要である。`beqz` も同様に、条件ごと消せる（分岐してもしなくても行き先が同じ）。コマ 5 の `if/else` 生成や、コマ 6 のループ生成で自然に生じる形である。

5.3 不動点まで繰り返す

ある置換で別の置換が可能になることがあるので、何も変わらなくなるまで繰り返す。この駆動部（`optimize`）は完成済みである。

補足

なぜテキストを書き換えるのか。本物のコンパイラは、アセンブリの文字列ではなく命令の内部表現に対してピープホールをかける。テキスト処理は「行の切り出し」という本質的でない仕事が増えるが、コンパイラ本体に一切手を入れずに追加できるという利点がある。まず効果を体感し、必要になったら内部表現に移すのは、実務でもよくある順序である。

6 実装

Step	実装対象	ファイル
1	<code>fold_binary</code> / <code>fold_unary</code>	<code>fold.py</code>
2	<code>fuse_push_const_pop</code>	<code>peephole.py</code>
3	<code>remove_jump_to_next</code> / <code>remove_branch_to_next</code>	<code>peephole.py</code>

走査・判定・駆動はすべて完成済みで、書くのは「規則そのもの」だけである。

6.1 動かし方

`optcc.py` は、自分の `final/mycc.py` に 2 つのパスを差し込んで実行するラッパーである (完成品。編集しない)。

```
# 最適化ありでコンパイル
python3 optcc.py ../../../../final/tests/f05_for.c

# パスを個別に外して比べる
python3 optcc.py --no-fold ../../../../final/tests/f05_for.c
python3 optcc.py --no-peephole ../../../../final/tests/f05_for.c

# 命令数を数える
python3 optcc.py ../../../../final/tests/f05_for.c | python3 ../count_insns.py
```

`peephole.py` は単体のフィルタとしても使える。

```
python3 ../../../../final/mycc.py foo.c | python3 peephole.py
```

7 テスト

```
python3 check.py      # Step ごとの単体テスト (未実装は SKIP)
python3 golden.py     # fixed15 全通の確認 + 命令数の before/after 表
```

`golden.py` は 2 つのことを続けて行う。

1. 正しさ: `fixed15` を最適化ありでコンパイルして全テスト実行
2. 効果: 各テストの命令数を最適化なし／ありで数えて表示

全テスト **PASS** かつ命令数が減っていることが、この回の完了条件である。参考までに、教員の実装では合計 1018 → 853 命令 (約 16% 減) になる。

8 発展課題

1. 代数的な単純化: $x * 1 \rightarrow x$ 、 $x + 0 \rightarrow x$ 、 $x * 0 \rightarrow 0$ を `fold_expr` に追加する。
 $x * 0$ はなぜ安全か（副作用がないか）も考える
2. `li` の直後の演算: `li a1, C + add a0, a1, a0` を `addi a0, a0, C` (C が 12bit に収まる時) に潰す規則を追加する
3. 到達不能コードの削除: `j` や `ret` の直後からラベルまでの命令は決して実行されない。
これを削除する規則を書く
4. 測る: `qemu-riscv64` を使って実行命令数（動的な数）を測り、静的な命令数の削減とどう違うか観察する。ループの中の 1 命令は何度も実行される

補足

コラム: 最適化の名前。定数畳み込み（constant folding）とピープホール（peephole optimization）は、どちらも 1960 年代から知られている古典的な手法である。この先には、共通部分式の除去、ループ不変式の移動、レジスタ割り当て（B2）、末尾呼び出し最適化（B3）などが続く。最適化は「賢いことをする」よりも「壊さない範囲で無駄を見つける」技術であり、そのために必要なのは、いま自分が持っているようなテストスイートである。